ADULTERA

Food Safety Evidence System

COMPLETE TECHNICAL AND CONCEPTUAL DOCUMENTATION

About this document

ADULTERA is a citizen-facing food adulteration detection and evidence-recording system. It teaches home chemical tests, scores risk, clusters reports by batch number across geographies, and preserves every case as a dated, geo-tagged, evidence-backed record mapped to the relevant FSS Act provisions.

The unit of insight is the batch, not the complaint.

Sections

1–4	Purpose, problem, idea, architecture
5–7	Frontend, backend, data model
8–10	Compliance, hosting, local development
11–14	Fixes, security, limitations, next steps

Documentation report · October 15, 2026

Contents

1. What the Website Actually Does 2
2. The Problem It Solves 2
3. How the Idea Was Found 3
4. Architecture 3
5. Frontend 4
6. Backend 5
7. Data Model 7
8. Compliance Module (/compliance) 7
9. Hosting & Infrastructure 8
10. Local Development 9
11. Problems Encountered and Fixed 9
12. Security Notes 10
13. Honest Limitations 10
14. Possible Next Steps 10

1. What the Website Actually Does

ADULTERA is a **citizen-facing food adulteration detection and evidence-recording system**. It has four connected jobs:

1. **Teach** — a library of 10 home-based chemical tests that detect common food adulterants (starch in milk, metanil yellow in turmeric, argemone oil in mustard oil, rhodamine B in chilli, vanaspati in ghee, and more). Each test lists reagents, step-by-step procedure, the expected colour for a pure sample, the colour for an adulterated sample, and safety warnings.
2. **Score** — a weighted risk engine that converts a raw home-test observation into a 0–100 risk score with a LOW / MODERATE / HIGH / CRITICAL band, and shows exactly which points came from which signal.
3. **Cluster** — a geospatial detection engine. Independent reports that share a **batch number** but come from **different areas or different citizens** are grouped into a cluster. This is the core idea: a single adulterated packet is a consumer problem; five reports of the same batch across three cities is a **supply-chain problem** that regulators can act on.
4. **Preserve** — every case becomes a dated, geo-tagged, evidence-backed record stored in PostgreSQL, viewable on a map and in an Evidence Vault.

The unit of insight is the batch, not the complaint. That is what separates this from a complaint portal.

2. The Problem It Solves

The problem

- Food adulteration is overwhelmingly **caught by consumers, not inspectors**. A regulator cannot sample every packet. A consumer can test the packet in front of them.
- But a consumer who finds adulteration has **no path to make it count**. They have a suspicion, not a record. There is no structured way to bind their observation to a batch number, a place, a date, and photographic evidence.
- Consequently, isolated reports are **statistically invisible**. One report of "bad turmeric" is noise. Twelve reports of batch `T24091` across Pune, Mumbai and Nashik is a **pattern with a traceable supply chain**.
- Meanwhile the consumer has no idea **what the law says**. They don't know that FSSAI Regulation 2.1.1(7) specifically prohibits selling turmeric containing any foreign substance, or that selling an adulterant injurious to health carries a penalty up to ₹10 lakh.

The solution

ADULTERA converts a private suspicion into a **citable, geo-distributed, evidence-backed batch signal**, and maps every finding back to the specific law it engages. The output of the system is not "a complaint" — it is **"batch X is adulterated, here are N independent witnesses across M areas, here is the photographic evidence, and here are the sections of the FSS Act that apply."**

That is a form a Food Safety Officer can actually act on.

3. How the Idea Was Found

The idea did not start from a technology. It started from a structural observation about **how adulteration actually spreads**.

Traditional thinking treats adulteration as a *quality* problem: the packet is bad. That framing produces a quality-control tool, and quality control is the wrong layer — it requires the manufacturer to cooperate, and by the time a lab tests a batch, the batch is already sold.

A different framing is to treat it as a **supply-chain observation problem**:

> *Adulteration is introduced upstream, at the point where a cheap substance is blended into a valuable one. That blending event has a **fingerprint** — a batch number. If that batch number recurs across distant geographies and unrelated consumers, the adulteration is systemic to that batch, not incidental to that shop.*

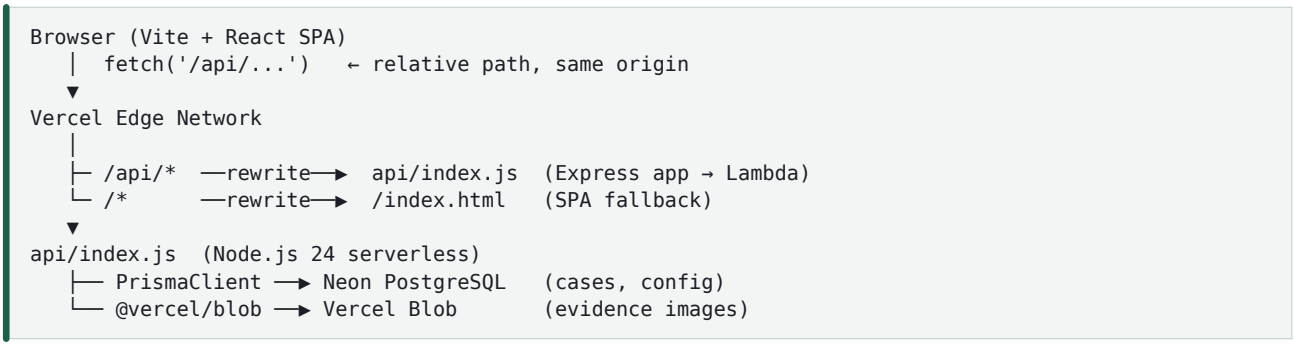
This yields three design commitments that follow directly:

Commitment	Reasoning
Batch number is the primary key of insight	It is the only token shared between the adulterated product and its source. Brand is too coarse (whole brand may be tainted), shop is too narrow, date is too noisy.
Geography must be a first-class signal	A single shop can serve one adulterated packet to one customer by accident. Five areas cannot, unless the batch itself is adulterated. This is why `minUniqueAreas` exists as a separate threshold from report count.
Evidence must be captured at test time, not reconstructed later	A home test is only meaningful with its colour reaction, its batch label, and its location. So `evidence`, `reactionDetails` and `location` are stored on the case record itself, not as a separate attachment flow.

Everything else in the codebase — the risk weights, the cluster threshold, the map, the evidence vault — follows from those three commitments.

The decision to centre **FSSAI's own published lab methods** as the authority for each test was made later, when it became clear that a home test is only a *screen*. It creates reasonable suspicion. Only a notified laboratory result is proof. Saying so explicitly is more credible than implying the home test is conclusive.

4. Architecture



The two-backend pattern

The project ships **two backends that share the same route logic and data modules**:

File	Used by	Why
`server/server.js`	`npm run dev` (localhost)	Long-lived Express server on port 3001. Falls back to **in-memory storage** if `DATABASE_URL` is absent, so the app is runnable with zero infrastructure.
`api/index.js`	Vercel production	Serverless function. Requires a real database.

Both import identical logic from `server/data/` and implement the same routes. This keeps local development frictionless while production runs on managed infrastructure.

5. Frontend

Stack

- **React 18** + **Vite 6**
- **Tailwind CSS 3.4** with a custom design-token palette (`surface`, `brand`, `danger`, `success`)
- **Leaflet 1.9** + **MapTiler** for mapping
- **lucide-react** for icons
- Hash-based routing (no react-router dependency)

Pages

Route	File	Purpose
`/`	`pages/Home.jsx`	Landing, system modules, hero
`/tests`	`pages/TestLibrary.jsx`	Browse all 10 tests
`/test/:id`	`pages/TestDetail.jsx`	Guided test walkthrough
`/report`	`pages/ReportCase.jsx`	File a report
`/vault`	`pages/EvidenceVault.jsx`	Browse stored evidence
`/map`	`pages/ClusterMap.jsx`	Cluster map + list toggle
`/risk`	`pages/RiskCalculator.jsx`	Interactive risk scoring
`/admin`	`pages/AdminConfig.jsx`	Tune thresholds and weights
`/compliance`	`pages/Compliance.jsx`	FSSAI regulatory reference

Shared components

`components/ui.jsx` exports `Spinner`, `LoadingState`, `EmptyState`, `ErrorState`, `StatCard`, `ProgressBar`, `Modal`, `Tabs`. `Layout.jsx` owns the sticky glass navbar and mobile drawer.

API layer — `src/api.js`

A single `request()` helper wraps `fetch`. It:

- prefixes every call with `API_BASE = '/api'` (relative, so the same build works on localhost and production),
- JSON-stringifies object bodies but passes `FormData` through untouched,
- distinguishes empty responses from errors,
- throws the server's `error` field into an `Error` so pages can render it.

All endpoints are centralised in the exported `api` object, so no component calls `fetch` directly.

Mapping — `ClusterMap.jsx`

- Creates the Leaflet map in a `useEffect` keyed on `[showMap, clusters, cases]`
- Destroys and recreates the instance on re-run to avoid the classic "Map container already initialised" error
- Draws a `circleMarker` per report, radius scaled by `reportsCount`, red when `isClusterAlert`
- Adds a translucent outer ring for alerts
- `fitBounds` with `pad(0.15)` on load
- **Tile provider is chosen at runtime:** MapTiler when `VITE_MAPTILER_KEY` is present, otherwise OpenStreetMap. Without this fallback the map renders blank grey when the key is absent.

6. Backend

Stack

- **Express 4** — routing, JSON body parsing
- **Prisma 6** — ORM over PostgreSQL
- **@vercel/blob 2.8** — evidence image storage (OIDC auth)
- **multer 2** — multipart upload parsing with MIME allow-listing
- **helmet** — security headers
- **cors** — origin control

Routes

```
GET    /api/health
GET    /api/tests                GET /api/tests/:id
GET    /api/cases                GET /api/cases/:id
POST   /api/cases                PATCH /api/cases/:id/status
GET    /api/clusters
POST   /api/risk-score/preview
GET    /api/config               POST /api/config
POST   /api/reset-demo
POST   /api/upload
GET    /api/fssai/reference
```

Risk scoring — `calculateRiskScore()`

Four weighted signals, capped at 100:

Signal	Default weight
Home test returned "suspected"	40
≥2 prior reports of the same product	20
≥2 prior reports of the same batch	15
Evidence attached	10

Thresholds: `≥80 CRITICAL`, `≥60 HIGH`, `≥35 MODERATE`, else `LOW`. Weights are editable at runtime via `/api/config` and stored as JSON.

Cluster detection — `detectBatchClusters()`

Groups cases by a composite key `PRODUCT:::BATCH` (uppercased, trimmed, `N/A` excluded). For each group it computes:

- `reportsCount`, `uniqueAreasCount`, `uniqueCitiesCount`, `uniqueUsersCount`
- a banner of flags: `isClusterAlert`, `clusterStatus`, `caseIds`

`isClusterAlert` requires **both** conditions:

```
reportsCount >= clusterThreshold  AND
(uniqueAreas >= minUniqueAreas  OR  uniqueUsers >= clusterThreshold)
```

This AND is the whole point of the product. Seven reports from one person in one lane is not a cluster. Three reports from three different areas is.

Vercel serverless specifics

`api/index.js` exports the Express `app` directly. Two non-obvious constraints:

1. `vercel.json` **rewrites must preserve the original path.** The SPA fallback must exclude `/api/`:

```
"rewrites": [
  { "source": "/api/(.*)", "destination": "/api" },
  { "source": "/((?!api/).*)", "destination": "/index.html" }
]
```

Vercel passes the original `req.url` to the function, so `app.get('/api/cases')` matches correctly. Removing the negative lookahead sends `/api/*` into the SPA and the API 404s.

2. **Do not pin** `functions` **runtime in** `vercel.json` unless the project's Node version matches it. A pinned `@vercel/node@3.0.1` against a Node 24 project fails the whole build.

Blob uploads

`@vercel/blob` ≥2 authenticates via Vercel's **OIDC** credentials automatically. The upload handler therefore passes **no** `token` option — supplying `process.env.BLOB_READ_WRITE_TOKEN` explicitly (as v0.x required) breaks it.

7. Data Model

```
model Case {
  id              String   @id            // "ADL-2026-00481"
  userId          String
  userName        String
  productName     String
  brand           String
  category        String
  batchNumber     String   // ← the clustering key
  mfgDate         String?
  expDate         String?
  purchaseDate    String?
  retailer        String?
  location        Json?    // { city, area, lat, lng, address }
  testId          String?  // links to standardTests
  finding         String?
  confidence      Int      @default(85)
  reactionDetails Json?    // { hue, saturation, dominantColor, matchedPixelsPct }
  evidence        Json?    // { hasProductPhoto, productPhotoUrl, ... }
  status          String   @default("Reported")
  riskScore       Int      @default(0)
  notes           String?
}

model Config {
  id               Int    @id @default(autoincrement())
  clusterThreshold Int    @default(3)
  minUniqueAreas   Int    @default(2)
  weights          Json
}
```

`location`, `evidence` and `reactionDetails` are `Json` because their shape varies by test and by which evidence types were captured.

8. Compliance Module (`/compliance`)

Added to ground every test in real law rather than folklore. All content is sourced from primary FSSAI / Government of India publications, and every claim on the page links to its official PDF.

Data — `server/data/fssaiReference.js`

Export	Contents
`legalFramework`	FSS Act 2006, key statutory definitions (§2 adulterant, unsafe, sub-standard, extraneous matter)
`penalties`	10 provisions — §30, §31, §32, §57, §57(1)(ii), §58, §59, §59(2), §65, §65A
`prohibitedSales`	14 prohibitions from the FSS (Prohibition and Restrictions on Sales) Regulations 2011, each mapped to a `relatedTest`
`colourRegulation`	100/200 ppm limits, 10 non-permitted colours, permitted INS examples
`officialLabMethods`	FSSAI *Manual of Methods of Testing and Analysis for Detection of Adulterants* — chapters II/III/IV, mapped to tests
`surveyData`	All-India Edible Oil Survey (Jan 2022): 4,433 samples, 0.34% argemone-oil positive
`productStandards`	Trans-fat ≤2%, hexane ≤0.5 ppm, argemone test must be negative
`sources`	7 official PDF links

Notable mappings

- `2.1.1(7)` "turmeric containing any foreign substance" → `turmeric-metanil-yellow`
- `2.1.1(6)` "vanaspati to which ghee has been added" → `ghee-vanaspati`
- `2.3.3` honey may not be marketed if not pure → `honey-invert-sugar`
- Manual Ch. III "Detection of Argemone Oil" (crimson-red in 20–30 s) → `oil-argemone`
- Manual Ch. IV "Detection of Metanil Yellow / Brick Powder" → chilli and turmeric tests

Why this matters

It converts the tool from a curiosity into something a **Food Safety Officer can cite**. It also honestly bounds the claim: the compliance page states that a home test is a *screen* producing reasonable suspicion, and that only a notified laboratory result is legal proof. That restraint makes the product more credible, not less.

9. Hosting & Infrastructure

Concern	Provider	Why
Hosting / CDN	**Vercel**	Zero-config SPA + Lambda from one repo; instant rollback
Database	**Neon** (via Vercel Marketplace)	Free tier, serverless Postgres, pooled connection string
Evidence storage	**Vercel Blob**	Native to Vercel, OIDC auth, public CDN URLs
Map tiles	**MapTiler**	Vector raster tiles; free tier with OSM fallback

Environment variables

Variable	Scope	Purpose
`DATABASE_URL`	Server	Neon pooled Postgres URL
`BLOB_READ_WRITE_TOKEN`	—	**Not needed** with `@vercel/blob` ≥2 + OIDC
`VITE_MAPTILER_KEY`	Public	MapTiler client key; absent → OSM fallback
`ALLOWED_ORIGINS`	Server	Comma-separated CORS allow-list; empty = allow all

`VITE_`-prefixed vars are **inlined into the public bundle** by Vite. MapTiler client keys are designed to be public, but treat any `VITE_` value as public — never put a secret behind that prefix.

Deploy

```
npm install
npx prisma migrate deploy   # or: prisma db push
npx vercel deploy --prod
```

Post-deploy, seed the database once:

```
node prisma/seed.js
```

10. Local Development

```
npm install
npm run dev          # server :3001 + client :5173
```

Vite proxies `/api` and `/uploads` to `localhost:3001`. Without `DATABASE_URL`, the local server logs `using in-memory storage` and runs fully — no infrastructure required to develop the UI.

Script	Action
`dev`	concurrently server + client
`server`	`node server/server.js`
`client`	`vite`
`build`	`prisma generate && vite build`
`prisma:generate` / `migrate` / `seed`	Prisma tasks
`postinstall`	`prisma generate` (so Vercel always has a client)

11. Problems Encountered and Fixed

Every item below was a real production failure, reproduced and verified fixed.

Symptom	Root cause	Fix
`Build Failed — Expected double-quoted property name, position 1098`	Trailing comma after `devDependencies` in `package.json`	Removed the trailing comma
`ERR_MODULE_NOT_FOUND: Cannot find package 'helter'`	Typo in `api/index.js` import (`helter` for `helmet`)	Corrected to `'helmet'`
API returned a Vercel login page	Deployment Protection enabled	Disabled Vercel Authentication
`"Failed to fetch cases"` on every DB route	`DATABASE_URL` never set in Vercel	Provisioned Neon, `prisma db push`, seeded
`/api/*` returned `bom1:: NOT_FOUND`	An over-aggressive rewrite removed the SPA fallback's negative lookahead	Restored `"/((?!api/).*)"`
Map rendered blank grey	`VITE_MAPTILER_KEY` unset → empty MapTiler key	Added OpenStreetMap runtime fallback + set the key
Uploads: `Vercel Blob: No token found`	`@vercel/blob` 0.x has no OIDC support	Upgraded to 2.8.2, removed explicit `token` option
Build failed: `Found invalid Node.js Version "24.x"`	`functions.runtime` pinned to `@vercel/node@3.0.1` (Node 18 only)	Removed the `functions` pin, let Vercel auto-detect

Process note. Several of these were self-inflicted during debugging — routing that was originally correct was changed, then reverted once the working baseline was re-established. The reliable method was: stop theorising, `curl` the live endpoint, compare against the last known-good deployment, and change one variable at a time.

12. Security Notes

- **Evidence upload**: MIME allow-list (JPEG/PNG/GIF/WebP only), 10 MB cap, server-generated filenames. Never trust `originalname`.
- **Helmet** sets standard security headers.
- **CORS** is origin-restrictable via `ALLOWED_ORIGINS`.
- **Prisma** uses parameterised queries — no string-built SQL anywhere in the codebase.
- **Secrets**: `DATABASE_URL` is server-only. Anything prefixed `VITE_` is compiled into public JS and must be treated as public.
- **Not yet implemented** (would be required before real-world deployment): rate limiting on report submission, CAPTCHA on public filing, Food Safety Officer authentication on `/admin`, and audit logging of status changes.

13. Honest Limitations

1. **Home tests are screens, not proof.** Colour reactions can be confounded by lighting, camera white balance, and the observer's device. The system says so explicitly.
2. **Batch numbers are self-reported.** A consumer may misread or mistype a batch code, which would fragment a genuine cluster. OCR-assisted batch capture would reduce this.
3. **No authentication.** Anyone can file a report, so the dataset is unauthenticated and could be gamed. Citizen sign-in and Food Safety Officer verification are the natural next steps.
4. **Cluster detection is naive.** It groups by an exact batch string. Near-misses (`T24091` vs `T-24091`) do not match. Fuzzy matching and OCR normalisation would help.
5. **Risk weights are heuristics**, not empirically calibrated against FSSAI lab outcomes.
6. **Seed data is synthetic** and clearly marked as demo data. No real enforcement statistics are presented as the system's own.

14. Possible Next Steps

- OCR batch numbers and expiry dates from the product photo at report time
- PWA install + offline test procedure for low-connectivity users
- Officer dashboard with jurisdiction-scoped cluster triage
- Multilingual test instructions (Hindi, Marathi) given the current geographic spread
- SMS/WhatsApp alerts when a batch a user already reported is later confirmed
- Integration with FSSAI's public complaint channel